

CSC 402 – Mobile Application Programming
Lab03 - Layouts, Modifiers and Material Design 3
Tasks 1–10

Student Name: Elaf Hassan Al-Nutayf

Student ID: 2240005931

Task 1 – Your First Modifier-Accepting Composable

A Student data class was created to store the student's name, program, GPA, email, and city. A reusable StudentCard composable was also created with a Modifier parameter placed after the required student parameter. At this stage, the composable body was intentionally left empty, so the Preview showed no card content.

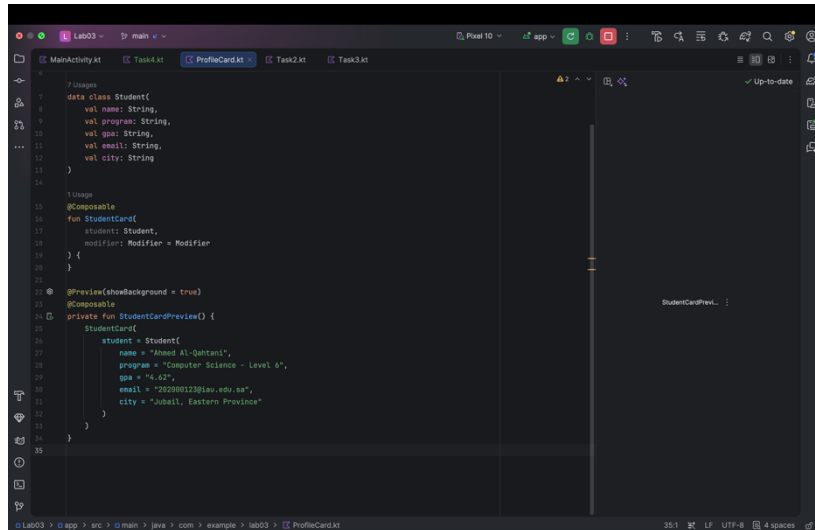


Figure 1. Task 1 StudentCard structure with the expected empty Preview.

Task 2 – Rebuild the Profile Card from the Picture

The student profile card was rebuilt from the provided target image using Jetpack Compose. Card, Column, Row, Text, HorizontalDivider, and Modifier were used to organize the student information. Modifier.weight(1f) was used in the Row so the GPA stays aligned on the right side of the card.

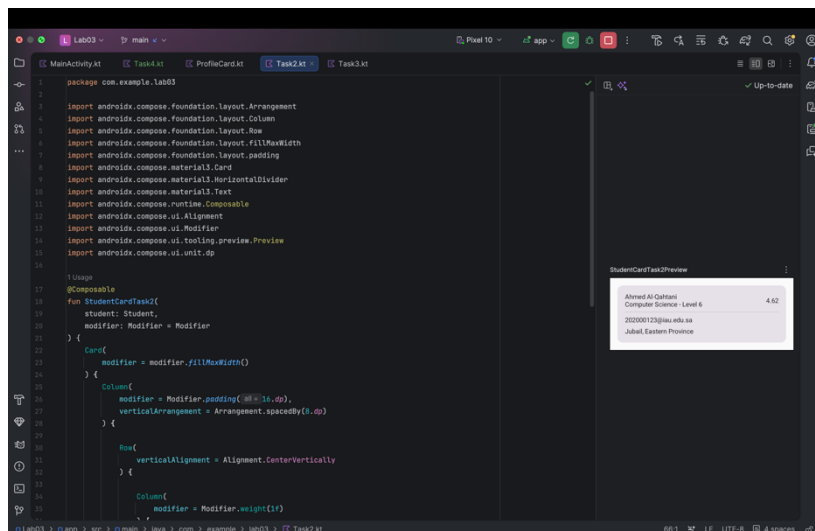


Figure 2. Student profile card created in Task 2.

Q1. Explain the difference between verticalArrangement and verticalAlignment and what would happen if the outer Column were replaced with a Row.

`verticalArrangement` controls how children are arranged along the vertical main axis of a `Column`, while `verticalAlignment` controls how children are aligned vertically inside a `Row`. In this card, the `Column` places the sections from top to bottom, while the `Row` vertically aligns the items within the same horizontal line. If the outer `Column` were replaced by a `Row` without changing anything else, the card sections would be placed horizontally instead of vertically and the layout would become compressed and incorrect.

Task 3 – Add a Box: Avatar with a Badge

An avatar was added to the left side of the student information using a `Box`. A circular shape was created with `CircleShape`, and the initials were placed in the center. A smaller status indicator was positioned at the bottom-right of the avatar by nesting another `Box` inside the avatar container.

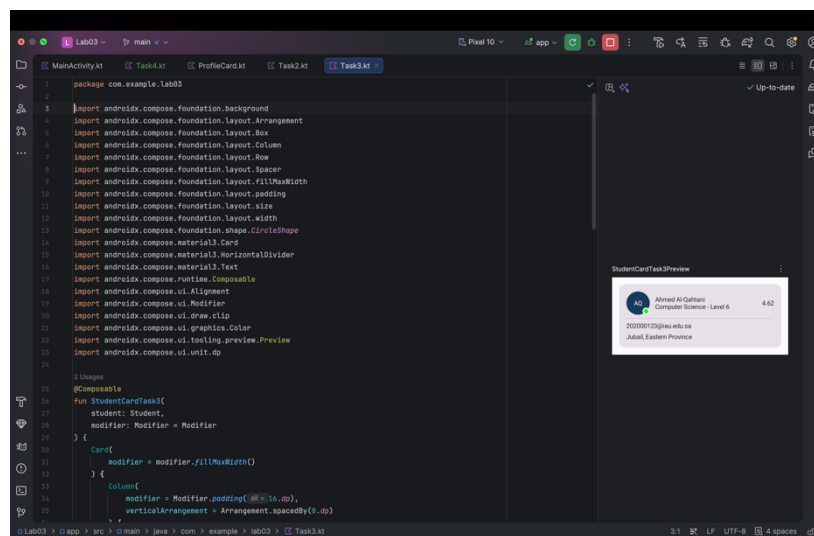


Figure 3. Student card after adding the avatar and status badge.

Task 4 – The Modifier Order Puzzle

Three modifier chains were tested to observe how modifier order affects layout and drawing. The prediction was recorded before comparing the output with the provided results.

Modifier Chain	Prediction	Verified Result
Chain A	Result 2	Result 2
Chain B	Result 1	Result 1
Chain C	Result 3	Result 3

The predictions matched the final results.

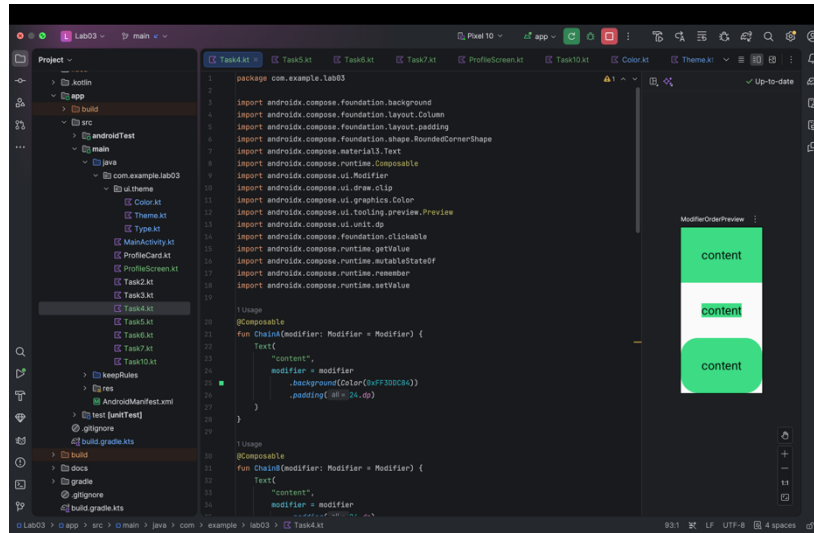


Figure 4. Modifier order results for Chains A, B, and C.

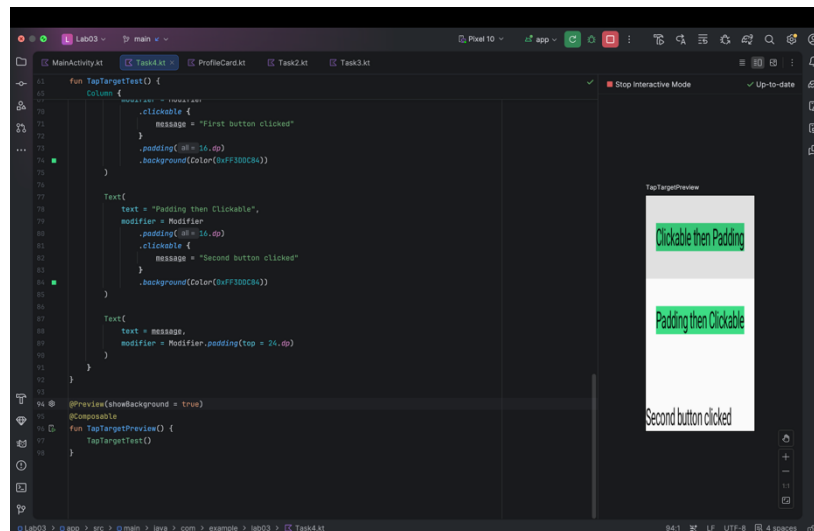
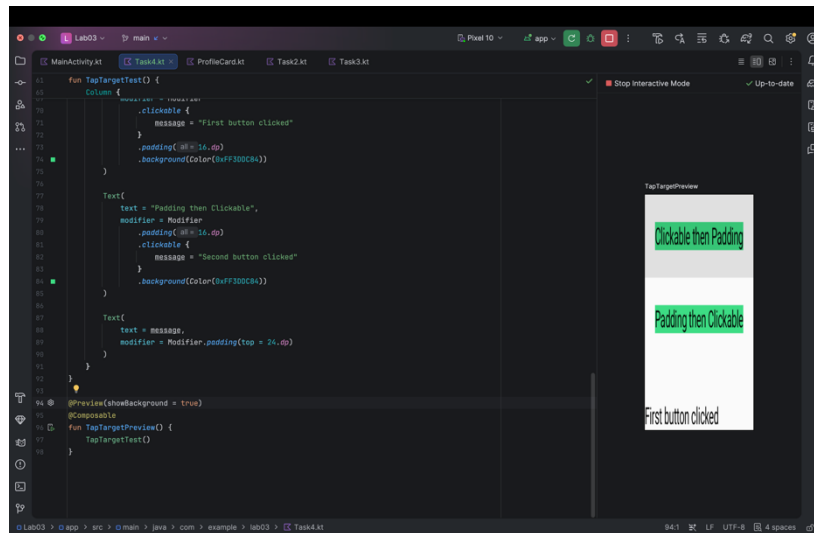


Figure 5. Tap-target test showing the different clickable areas caused by modifier order.

Q2. My prediction was Chain A → Result 2, Chain B → Result 1, and Chain C → Result 3, and the verified results matched the prediction. During composition, Compose creates the UI structure; during

layout, modifiers such as padding affect the size and position of the element; and during drawing, modifiers such as background and clip affect how that area is rendered. Because modifiers are applied in sequence, changing their order changes which area is padded, drawn, or clipped.

Q3. A composable function returns Unit because Compose uses a declarative UI model rather than returning a mutable widget object. If Text returned a widget object, the application would need to keep a reference to that object and manually modify its properties when the UI changed. In Compose, state changes trigger recomposition instead. The composable code belongs to the application layer of the Android software stack.

Task 5 – Colour Roles Instead of Hex Values

The hard-coded colors in the student card were replaced with semantic Material 3 color roles from MaterialTheme.colorScheme. The name uses onSurface, the secondary student information uses onSurfaceVariant, and the GPA uses primary. This allows the card colors to adapt automatically when the application theme changes.

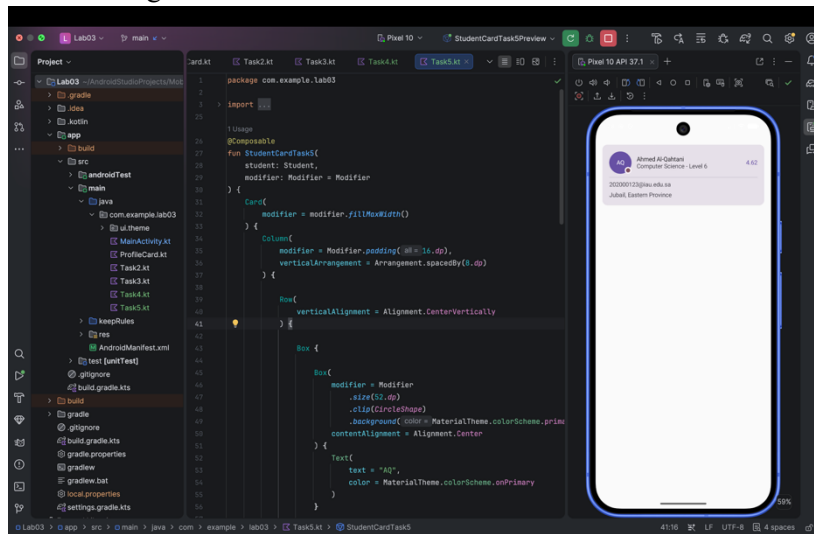


Figure 6. Student card after replacing hard-coded colors with Material color roles.

Task 6 – The Type Scale Instead of Raw sp

Hard-coded text sizes were replaced with styles from MaterialTheme.typography. The student's name uses titleLarge, the GPA uses titleLarge, the program uses bodyMedium, and the email and city use bodyMedium. Using the Material type scale keeps text styling consistent with the application theme.

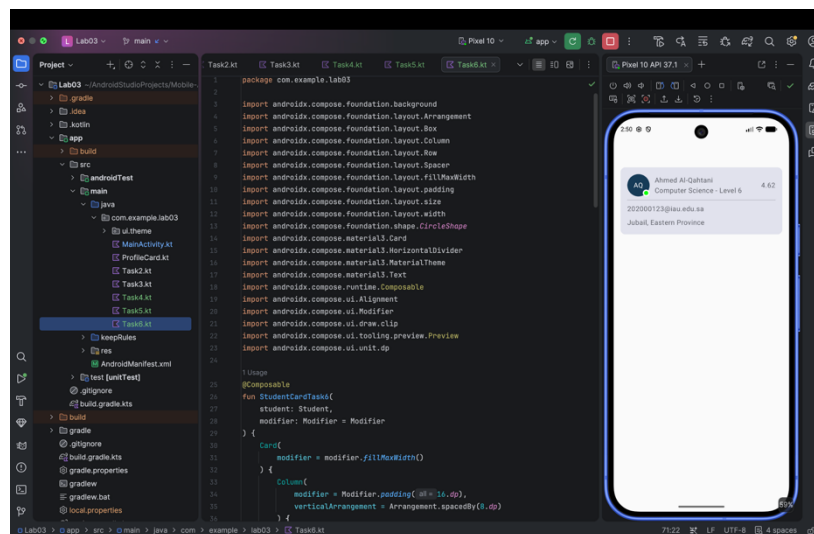


Figure 7. Final student card after applying Material typography.

Task 7 – Dark Mode Preview

Two previews were created for the same student card: one using light mode and one using dark mode. The card adapted correctly to both themes because the colors and typography were taken from MaterialTheme instead of being hard-coded.

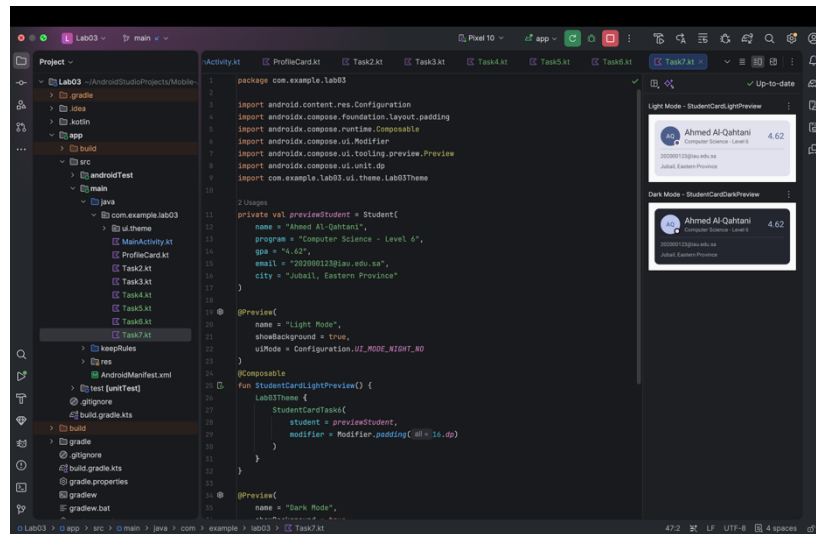


Figure 8. Student card rendered in light and dark mode.

Q4. Nothing broke when the card was first switched to dark mode because the card uses Material color roles instead of fixed colors. To verify this, one Material color was temporarily replaced with a hard-coded color. In the dark preview, that element no longer adapted correctly and its contrast became poor. The hard-coded color was then removed and the Material color role was restored.

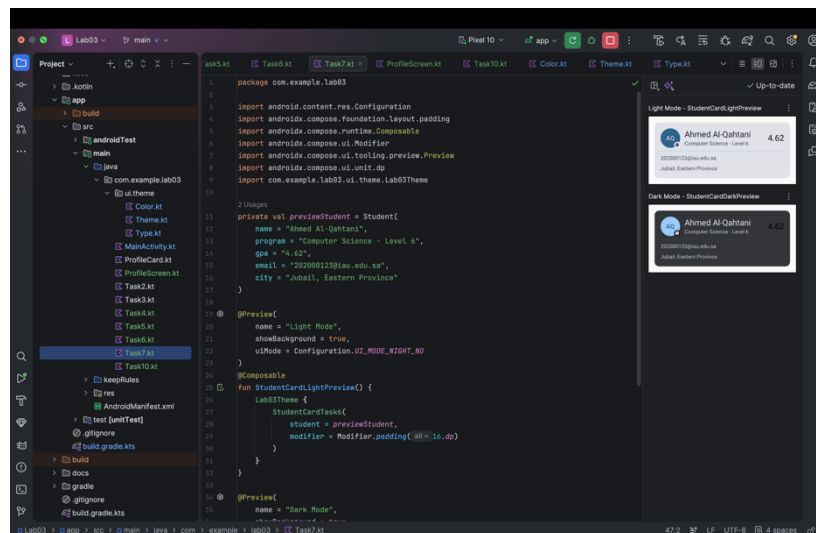


Figure 9. Dark-mode test after temporarily applying a hard-coded color.

Task 8 – Scaffold and TopAppBar

The student card was placed inside a ProfileScreen using Scaffold. A TopAppBar titled "My Profile" and a FloatingActionButton with an edit icon were added. The task was tested first without applying Scaffold's innerPadding and then again after applying it correctly.

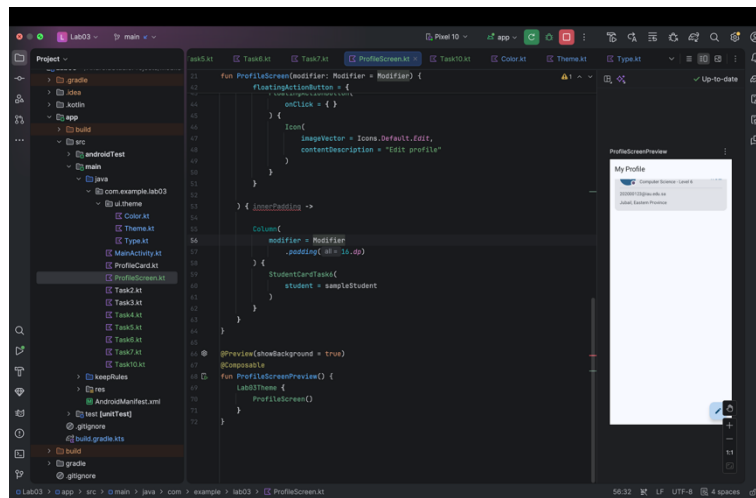


Figure 10. ProfileScreen before applying Scaffold innerPadding.

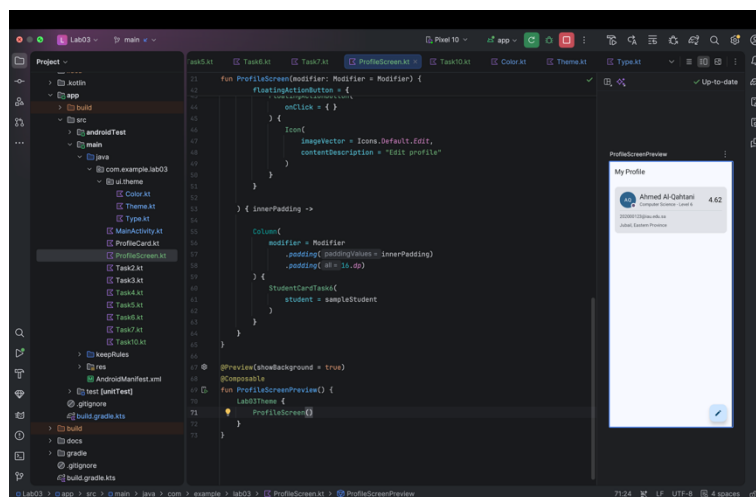


Figure 11. ProfileScreen after applying Scaffold innerPadding correctly.

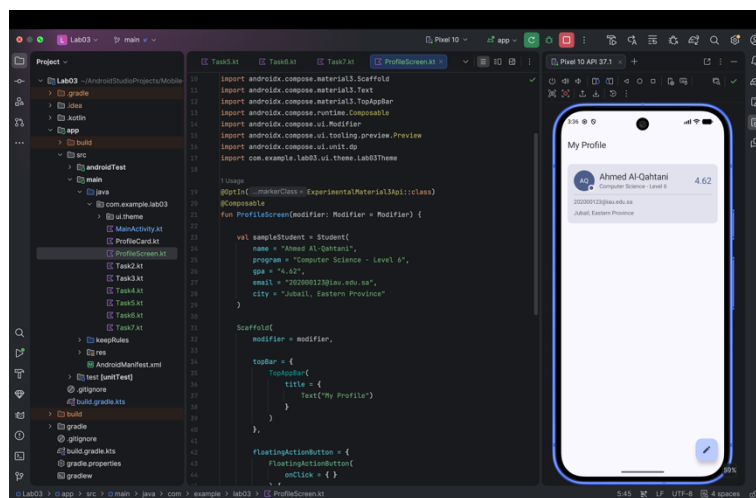


Figure 12. ProfileScreen running successfully on the Android emulator.

Q5. When `innerPadding` was ignored, the content could enter the space reserved for Scaffold components such as the `TopAppBar`. Scaffold provides `PaddingValues` that describe the space occupied by its UI slots. It cannot apply those values directly because the content composable controls its own layout, so the provided `innerPadding` must be applied using a `Modifier`.

Task 9 – Your Team's Theme

A custom Material 3 theme was generated and integrated into the project. The selected source color was `#173B57`, and the exported theme files were used to replace the default theme colors. Dynamic color was disabled so the application keeps the selected team color scheme consistently.

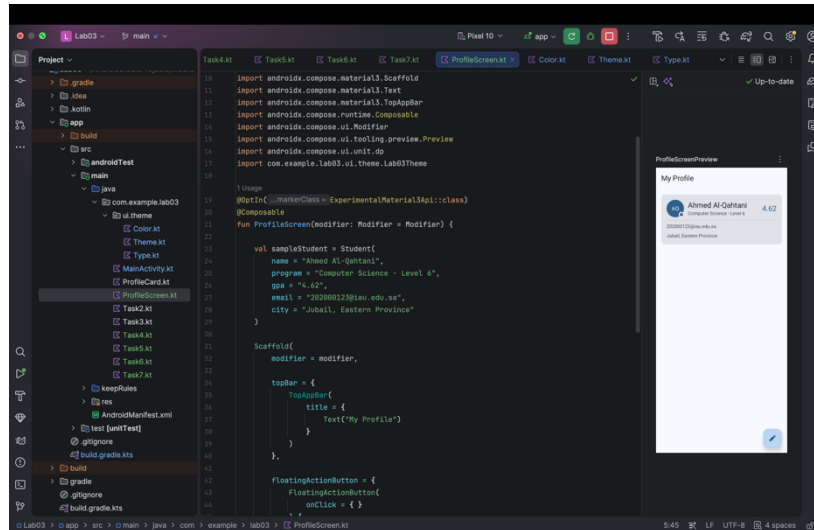


Figure 13. Student profile using the custom Material 3 theme.

Q6. Our team's source color is `#173B57`. We selected this dark blue because it represents security, privacy, and trust and matches the purpose of Digital Shadow. The `StudentCard` code did not need to change when the theme changed because Task 5 replaced fixed colors with semantic `MaterialTheme.colorScheme` roles and Task 6 replaced fixed text sizes with `MaterialTheme.typography` styles.

Task 10 – Challenge: From One Card to a Screen

A list of five Student objects was created and displayed using LazyColumn. Each student is rendered using the same reusable StudentCardTask6 composable. The LazyColumn uses `contentPadding` and `Arrangement.spacedBy(12.dp)`, while a header above the list displays "Students" and the total number of students. Light and dark previews were created to verify that the complete screen works correctly in both themes.

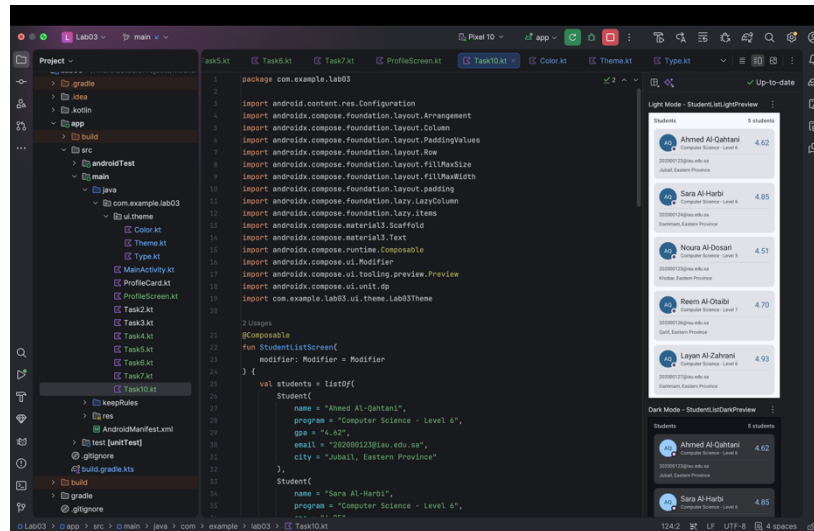


Figure 14. Final student list screen rendered in light and dark mode..

AI Use & Disclosure Log

Tool	How it was used	What I changed / verified
ChatGPT	Used as a pair programmer to explain requirements, suggest code structure, and troubleshoot Android Studio issues.	I reviewed the code, tested it on the emulator, adjusted layout and spacing, and verified the final output.

AI assistance was used as a pair-programming and explanation tool. All code was reviewed, tested, and verified in Android Studio before submission.